

Discrete forward, soft backward for differentiable lookup tables

Sign-pack lookup, top-2 ($n_{\text{alt}}=1$) blend, full- K soft surrogate

1 Setup

API surface. TinyMultiHeadLut is a layer

$$F : \mathbb{R}^E \rightarrow \mathbb{R}^{H \times D}, \quad x \mapsto y_h = \sum_{s=1}^{\text{tph}} \text{Lut}_{h,s}(x),$$

parameterised by an input width E , an output width D per head, a number of heads H , a number of tables per head tph , and a number of anchor pairs N per table (the LUT’s NAP). The layer holds $H \cdot \text{tph}$ independent lookup tables; each one maps x to a D -vector, and the per-head output sums the tph tables in that head. Two scalar temperatures shared by all tables, stored in log-space and exponentiated on use: T_{soft} (per-anchor sign sharpness) and T_{sel} (row-selection sharpness).

The math below describes a single table $\text{Lut}_{h,s}$ — every formula runs in parallel on all $H \cdot \text{tph}$ of them with their own (a_j, b_j) and W but shared $T_{\text{soft}}, T_{\text{sel}}$.

Anchor pairs and weight table. Each table fixes N anchor pairs $(a_j, b_j)_{j=1}^N$ once at module init and never moves them; the pair differences are

$$d_j = x_{a_j} - x_{b_j}, \quad D : \mathbb{R}^E \rightarrow \mathbb{R}^N, \quad (Dx)_j = d_j, \quad \text{rows}(D) = e_{a_j} - e_{b_j}.$$

A weight table $W \in \mathbb{R}^{K \times D}$ with $K = 2^N$ rows indexed by $b \in \{0, 1\}^N$.

Bit parity:

$$\chi_j(b) := 2b_j - 1 \in \{-1, +1\}.$$

Per-anchor soft sign and row score. For each anchor pair j and each row b define

$$p_j = \frac{d_j}{T_{\text{soft}} + |d_j|} = \text{sign}(d_j) \frac{|d_j|}{T_{\text{soft}} + |d_j|} \in (-1, +1), \quad \mathcal{S}(b) = \sum_{j=1}^N p_j \chi_j(b),$$

and the softmax distribution over rows

$$z(b) = \mathcal{S}(b)/T_{\text{sel}}, \quad \pi(b) = \text{softmax}_b z(b).$$

p_j is the smooth sign; π is the full- K softmax used by the backward surrogate (Section 3).

Main row. The argmax of $\mathcal{S}$ is the hard sign-pack:

$$b^* = \arg \max_b \mathcal{S}(b), \quad b_j^* = [d_j > 0].$$

Computing b^* takes N comparisons and packs the bits into one integer in $\{0, \dots, K-1\}$.

2 Two forward modes

Hard (`forward_mode='hard'`). The published default. Output is the single row picked by the sign-pack:

$$y^{\text{hard}} = W[b^*].$$

Cost: $O(N)$ to compute b^* , one row gather. No softmax, no row blend; temperatures play no role in y^{hard} .

Hybrid-smooth (`forward_mode='hybrid_smooth'`). An exact two-row softmax restricted to b^* and its least-confident Hamming-1 neighbour. Let

$$j^* = \arg \min_j |d_j|, \quad b_{\text{alt}} = b^* \oplus e_{j^*}.$$

Any single-bit flip of b^* yields a row whose score differs from $\mathcal{S}(b^*)$ by exactly $2|p_j|$. The smallest such gap is at $j = j^*$, so b_{alt} is the second-best row under $\mathcal{S}$.

Setting $d_{\min} := |d_{j^*}|$, the exact score gap is

$$\Delta(b^*, b_{\text{alt}}) = \mathcal{S}(b^*) - \mathcal{S}(b_{\text{alt}}) = 2|p_{j^*}| = \frac{2 d_{\min}}{T_{\text{soft}} + d_{\min}},$$

and the softmax of z restricted to $\{b^*, b_{\text{alt}}\}$ places mass

$$u = \sigma(-\Delta/T_{\text{sel}}) = \frac{1}{1 + \exp(\Delta/T_{\text{sel}})} \in (0, \tfrac{1}{2}]$$

on the alt row. The output is the two-row blend:

$$y^{\text{hyb}} = (1 - u) W[b^*] + u W[b_{\text{alt}}].$$

Both temperatures enter u : T_{soft} shapes Δ through p_{j^*} ; T_{sel} rescales the resulting log-odds. Cost: $O(N)$ plus two row gathers and one blend; no materialisation of $\pi \in \mathbb{R}^K$.

Limit. As $|d_{j^*}|/T_{\text{soft}} \rightarrow \infty$ or $T_{\text{sel}} \rightarrow 0^+$, $u \rightarrow 0$ and $y^{\text{hyb}} \rightarrow y^{\text{hard}}$. Hybrid-smooth is a strict refinement of hard: same b^* , extra mass u on the second-best row at the least-confident anchor.

3 Soft surrogate backward

Let $g := \partial L / \partial y \in \mathbb{R}^D$. The backward decomposes into two pieces:

- (A) *Weight gradient* flows *only* through the row(s) that actually contributed to y — one row in hard mode, two rows in hybrid-smooth mode.
- (B) *Input and temperature gradients* are computed as the analytic gradient of the full- K soft forward $y_{\text{full}} = \sum_b \pi(b) W[b]$ from Section 1, ignoring the truncation the actual forward applied. Identical in both forward modes.

(A) Weight gradient (mode-dependent)

Hard mode. Differentiating $y^{\text{hard}} = W[b^*]$ at fixed b^* :

$$\frac{\partial L}{\partial W[r]}^{(\text{hard})} = g \mathbf{1}[r = b^*]; \quad \text{all other rows } 0.$$

A one-row scatter into W per sample.

Hybrid-smooth mode. Differentiating $y^{\text{hyb}} = (1 - u)W[b^\star] + uW[b_{\text{alt}}]$ at fixed $u, b^\star, b_{\text{alt}}$:

$$\frac{\partial L}{\partial W[r]}^{(\text{hyb})} = (1 - u)g\mathbf{1}[r = b^\star] + u g\mathbf{1}[r = b_{\text{alt}}]; \quad \text{all other rows 0.}$$

A two-row scatter per sample: weight $(1 - u_b)g_b$ at row $b_b^\star$, weight $u_b g_b$ at row $b_{\text{alt},b}$.

In both cases this is the honest derivative of the truncated forward — no soft attribution to rows the forward did not touch. As $u \rightarrow 0$ the hybrid-smooth weight grad collapses to the hard weight grad, consistent with the forward limit.

(B) Input & temperature gradients via the full-soft surrogate

The forward used only $\{b^\star\}$ or $\{b^\star, b_{\text{alt}}\}$, but for $\partial L/\partial x$ and $\partial L/\partial T$ we want a denser local signal that reflects the full neighbourhood of $b^\star$. So we backpropagate as if the forward had been the full- K soft mode of Section 1, $y_{\text{full}} = \sum_b \pi(b)W[b]$, reusing the same $p_j, \mathcal{S}(b), z(b), \pi(b)$ defined there.

Through the softmax.

$$\begin{aligned} g_\pi(b) &= g^\top W[b] && \in \mathbb{R}^K, \\ \langle g_\pi, \pi \rangle &= \sum_b g_\pi(b) \pi(b), \\ g_z(b) &= \pi(b) (g_\pi(b) - \langle g_\pi, \pi \rangle) && (\text{softmax Jacobian}), \\ g_{\mathcal{S}}(b) &= g_z(b)/T_{\text{sel}}. \end{aligned}$$

Through the row-score sum. $\mathcal{S}$ is linear in p_j with coefficient $\chi_j(\cdot)$, so

$$g_{p_j} = \sum_b g_{\mathcal{S}}(b) \chi_j(b).$$

Through the soft sign. $p_j = d_j/(T_{\text{soft}} + |d_j|)$, so differentiating directly,

$$\frac{\partial p_j}{\partial d_j} = \frac{T_{\text{soft}}}{(T_{\text{soft}} + |d_j|)^2}, \quad g_{d_j} = g_{p_j} \frac{T_{\text{soft}}}{(T_{\text{soft}} + |d_j|)^2}.$$

Through D .

$$\frac{\partial L}{\partial x} = D^\top g_d, \quad \text{i.e.} \quad (\partial L/\partial x)_{a_j} = g_{d_j}, \quad (\partial L/\partial x)_{b_j} = -g_{d_j}.$$

Temperatures (log-parameterised, $\partial/\partial \log T = T \cdot \partial/\partial T$). z depends on T_{sel} only via the rescale $z = \mathcal{S}/T_{\text{sel}}$, and on T_{soft} only via p_j (hence through d_j in the same way as x):

$$\boxed{\frac{\partial L}{\partial \log T_{\text{sel}}} = -\sum_b g_z(b) z(b), \quad \frac{\partial L}{\partial \log T_{\text{soft}}} = -\sum_j g_{d_j} d_j.}$$

The temperature gradients are nonzero even in hard forward mode, where the temperatures had no effect on y at all — the surrogate gives them an update through the soft-pipeline Jacobian.

4 Why the asymmetry

Weight grad is hard because the forward is hard. The forward only ever reads $W[b^*]$ (hard) or $\{W[b^*], W[b_{\text{alt}}]\}$ (hybrid-smooth); any soft attribution to the remaining $K-1$ or $K-2$ rows would be a phantom update with no matching forward contribution. Restricting the weight grad to the rows the forward actually touched keeps the gradient an unbiased derivative of the actual computation.

Input/temperature grad is soft because we want a richer signal. x enters the forward only through b^* (hard) or through b^* , b_{alt} , and u (hybrid-smooth). The honest derivative w.r.t. x is therefore either identically zero (hard: b^* is a discrete function of x , the gather is x -independent at the chosen index) or dominated by a single coordinate (hybrid-smooth: only the j^* -th anchor pair feeds u). Neither is a usable training signal. The full-soft surrogate restores a dense signal at all N anchor pairs and gives the temperatures a useful update each step, at the cost of not being the literal Jacobian of the truncated forward. Empirically this trade — honest weight update, smooth input update — is what makes both forward modes trainable end-to-end while leaving inference as a sign-pack lookup.